Latency Sensitive Workload Tuning Guide

Release 1.0

Intel, Red Hat

Oct 19, 2026

CONTENTS:

1 Authors **3**

2 Revisions **5**

3 Assumptions **7**

4 Scope **9**

5 Intended Audience **11**

6 Problem Statement **13**
6.1 Overview . . . 13
6.2 System Level Tunable Layers . . . 14

7 Background Information **17**

8 KPI Definitions **23**
8.1 Abstract KPIs . . . 23

9 Measurement Tools **25**

10 Initial System Setup **27**
10.1 Hardware BOM . . . 27
10.2 BIOS Configuration . . . 27
10.3 Software BOM . . . 28
10.4 CaaS Deployment . . . 28
10.5 Performance Profile . . . 28

11 Measurement Tool Setup **31**
11.1 Hwlatdetect . . . 31
11.2 Cyclictest . . . 32
11.3 Oslat . . . 33

12 Sample Benchmark Results **35**
12.1 Hwlatdetect . . . 35
12.2 Hwnoise . . . 36
12.3 Cyclictest . . . 36
12.4 Oslat . . . 36
12.5 Timerlat . . . 37

13 Conclusion **39**

14 References **41**
14.1 Timing and Synchronization Standards 41
14.2 Transport and Latency Budget Standards 41
14.3 Best Practice and Industry Implementation Guidance 42
14.4 Typical Timing and Latency Targets for 5G RAN 42

15 Appendix **43**
15.1 Hwnoise 43
15.2 Timerlat 44

The following presents guidance for the necessary system level tunings along with the set of validations required for deploying and running latency sensitive workloads on a given Intel based platform.

CHAPTER

ONE

AUTHORS

- Timothy Miskell,Cloud Software Architect,Intel
- Ie Xiann Goh,Software Optimization Engineer,Intel
- Burla Balendu,Software Engineer,Intel
- Sam Diep,Wireless Systems Engineer,Intel
- Mike Beadle,Director Standard Wireless Access,Intel
- William Penner,Senior Principal Engineer,Intel
- Abel Chacko,Platform Application Engineer,Intel
- Dave Cain,Senior Principal Engineer,Red Hat
- Heikki Heliste,Manager,Red Hat
- Bart Wensley,Senior Principal Software Engineer,Red Hat
- Petri Hattula,Principal Technical Program Manager,Red Hat
- Brent Rowsell,Senior Principal Software Engineer,Red Hat
- Costa Shulyupin,Principal Software Engineer,Red Hat

CHAPTER

TWO

REVISIONS

Version	Description
1.0	Initial release

CHAPTER

THREE

ASSUMPTIONS

This guide is written with the following baseline assumptions regarding the operating system, subscription and system configuration:

1. **Operating System and Kernel**
    - The target system should be running Red Hat OpenShift Container Platform (RHOCP).
    - Real-time testing is performed on nodes / systems where a real-time kernel is installed and active.
    - You have administrator-level / cluster admin privileges on RHOCP.
2. **System configuration**
    - System BIOS is tuned to Intel, Red Hat and / or OEM recommendations for real-time workloads.

The following table presents the key term definitions that will be used throughout this document.

Term	Definition
5GNR	5G Next Generation Radio
ATIS	Alliance for Telecommunications Industry Solutions
BIOS	Basic Input/Output System
BOM	Bill of Materials
C-RAN	Cloud/Centralized Radio Access Network
C-State	An individual idle residency for a given CPU core
CaaS	Container-as-a-Service
CNF	Containerized Network Function
CPU	Central Processing Unit
CU	Centralized RAN Unit
DIMM	Dual Inline Memory Module
DPC	DIMM per Channel
DPDK	Data Plane Development Kit
DRAM	Dynamic Random-Access Memory
DU	Distributed RAN Unit
DUT	Device Under Test
EFI	Extensible Firmware Interface
ESMC	Ethernet Synchronization Messaging Channel
FHGW	Front-haul Gateway
GbE	Gigabit Ethernet
gNB	Base Station
GPS	Global Positioning System
GNSS	Global Navigation Satellite System
grub	GRand Unified Bootloader
HT	Hyper-thread

continues on next page

Table 1 – continued from previous page

Term	Definition
HW	Hardware
IEEE	Institute for Electrical and Electronics Engineers
IOMMU	Input-Output Memory Management Unit
IoT	Internet of Things
IRQ	Interrupt Request
ISV	Independent Software Vendor
ITU	International Telecommunication Union
KPI	Key Performance Indicator
LOM	LAN on Motherboard
NMI	Non-Maskable Interrupt
NF	Network Function
NFV	Network Function Virtualization
NUMA	Non-Uniform Memory Access
MIMO	Multi-Input and Multi-Output
MSI-X	Message Signaled Interrupt-Extended
OEM	Original Equipment Manufacturer
O-RAN	Open Radio Access Network
OCP	Open Compute Platform
OS	Operating System
OSV	Operating System Vendor
P-State	An individual frequency bin for a given CPU core
PDV	Packet Delay Variation
PHC	PTP Hardware Clock
PPB	Parts per Billion
PRTC	Primary Reference Time Clock
PPS	Pulse per Second
PTP	Precision Timing Protocol
SDN	Software Defined Networking
SoC	System on a Chip
SI	Solution Integrator
SMI	System Management Interrupt
SyncE	Synchronous Ethernet
RAS	Reliability; Availability and Serviceability
RAN	Radio Access Network
RDC	Resource and Documentation Center
RHOCP	Red Hat OpenShift Container Platform
RT	Real-time
RU	Radio Unit
SI	Solution Integrator
SLA	Service Layer Agreement
SW	Software
TDD	Time Division Duplex
TTI	Transmission Time Interval
VFIO	Virtual Function Input/Output
vRAN	Virtualized Radio Access Network
VNF	Virtualized Network Function

CHAPTER

FOUR

SCOPE

The intended scope and goals for this guide includes the following:

1. To deliver a standalone technical white paper that provides in depth guidance on how to setup and tune Intel x86 platforms to run latency sensitive workloads on top of Container-as-a-Service deployments, such as Red Hat OpenShift Container Platform.
2. The focus will be primarily upon Telecommunications workloads, such as Virtualized Radio Access Networks (vRAN) applications. However, the methodology and tunings provided within this document may also be applicable to other mission critical workloads, for example those within industrial IoT or financial sectors.
3. The documentation will take a ground-up approach illustrating how to configure the platform, the BIOS, the operating system, up to the CaaS layer.
4. This guide will not provide instructions for how to tune individual ISV applications, however, where relevant a suitable proxy workload, for example the Intel® FlexRAN™ application, will be referenced whenever a concrete example is needed of a latency sensitive workload.
5. Although specific tunings will not be provided for individual OEM platforms, general guidelines will be provided that apply generally across multiple server product lines.
6. Similarly, although specific tunings will not be provided for individual OS and CaaS layers, general guidelines will be provided that apply generally across multiple Linux based OS and CaaS variants.

CHAPTER

FIVE

INTENDED AUDIENCE

The target audience for this publication includes Original Equipment Manufacturer (OEM) vendors, Solution Integrators (SIs), Operating System Vendors (OSVs), Container-as-a-Service (CaaS) providers, along with Independent Software Vendors (ISVs) that contribute to the deployment and development of solutions in the space of latency sensitive workloads, for example RAN applications. The first iteration of this publication will be as a standalone document to be made available under NDA from Intel Resource and Documentation Center (RDC) for customers and providers within the area of Telecommunications (Telco) supporting such latency sensitive workloads.

CHAPTER

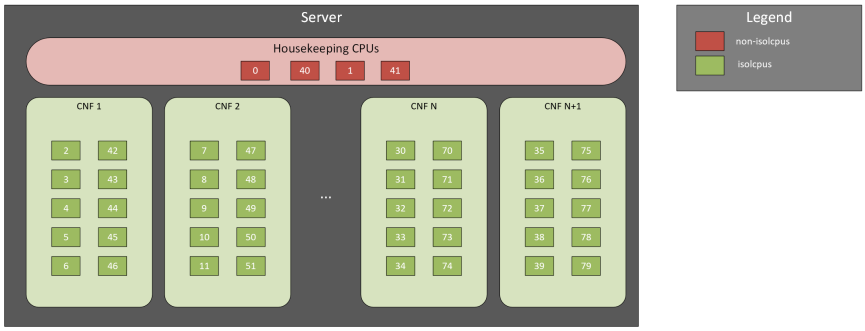
SIX

PROBLEM STATEMENT

6.1 Overview

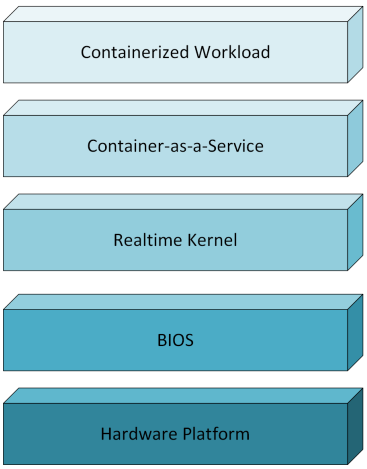
In order to explore the challenges associated with deploying latency sensitive workloads in a Software Defined Network (SDN) on general purpose hardware in a deterministic fashion, an example use case is provided to walkthrough the overall process. Traditionally, a mission critical workload, such as a RAN application, would be deployed individually on a fixed function hardware appliance where a single tenant has sole ownership of the underlying hardware resources. However, with the advent of SDN, virtualization technology, and the desire to reduce overall Capital Expenditure (CapEx) and Operational Expenditure (OpEx), such Network Functions have transitioned to Virtualized Network Functions (VNFs) and Containerized Network Functions (CNFs) deployed in a multi-tenant fashion on commodity hardware. As part of this transition, the underlying multipurpose infrastructure, which previously would have been fixed function, no longer is guaranteed to meet pre-defined Key Performance Indicators (KPIs) in a deterministic fashion. Instead the system consists of a set of shared resources that must be tuned accordingly to a given tenant workload.

The following figure presents an example core layout for an Intel x86 CPU that supports HyperThreading. Notice that as part of this core layout a subset of the total available cores are reserved for the host operating system, while the remaining cores are reserved for the multi-tenant workloads to be deployed on the platform. In this case, the cores that are assigned to the operating system are part of the non-isolated CPU list, or non-isolcpus. These "housekeeping" threads handle the set of standard kernel related tasks, such as scheduling processes, allocating or freeing memory, servicing interrupts, and so forth. The cores that are assigned to the multi-tenant workloads are part of the isolated CPU list, or isolcpus. These application threads handle the tasks associated with the mission critical workloads. In general, the goal when tuning a system for latency sensitive workloads is to ensure that the resources, such as the application threads, are able to service their specific tasks within a given narrow window of time. For a vRAN application this is often with reference to the Transmission Time Interval (TTI), in which case any additional or potentially unrelated tasks that a given workload thread must handle subtracts from the overall allotted time for servicing a given TTI interval. In the context of vRAN, the TTI interval for a Narrowband use case is typically 1,000 microseconds, while for a MMIMO use case the TTI interval is typically 500 microseconds. The methodologies provided within this document are focused upon reducing the percentage of time spent by a given thread servicing tasks unrelated to meeting the target TTI interval, for example handling System Management Interrupts (SMIs).



6.2 System Level Tunable Layers

The following figure presents an overview of the individual layers present within the Device Under Test (DUT). For each of these layers a set of tunings are applied in order to ensure that the system performs in a deterministic manner with the goal of meeting the required Service Level Agreement (SLA) defined by the workload.



Beginning with the foundational, the platform provided by the OEM manufacturer should be populated with a set of hardware ingredients designed to meet the stringent requirements for low latency application. For example, for RAN applications the specific CPU selected should support configurable P-states along with C-states, the AVX512 instruction set, and include a set of accelerators such as vRAN Boost for Forward Error Correction (FEC) hardware offload. The platform should allow populating the CPU in a single socket fashion in order to avoid the potential overhead associated with Non-Uniform Memory Access (NUMA). In addition, the memory channels in the platform should be populated in a single DIMM per Channel (DPC) fashion using dual rank memory modules with 8 banks (2Rx8). The platform should also support hardware offload of Precision Time Protocol (PTP) timestamping as well as a PTP Hardware Clock (PHC). The support for PTP hardware offload can be in terms of an integrated ethernet adapter, such as the Intel ® E825 Ethernet Controller, or a discrete ethernet adapter, such as the Intel ® E830 Ethernet Adapter. Furthermore, the PHC can be implemented with the aid of a Digital Phase-Locked Loop (DPLL) discrete timing module, or as before as part of a discrete ethernet adapter. Optionally, the system can include support for connecting to a Global Positioning System (GPS) uplink that provides constellation and timing information by means of a Global Navigation Satellite System (GNSS) discrete module.

Proceeding to the Built-In Operating System (BIOS), the hardware platform provided by the OEM manufacturer should either (1) provide a workload profile specifically designed for applications such as RAN or (2) expose all of the necessary configurable settings required to meet the timing requirements for a latency sensitive application. For example, the BIOS should allow locking the uncore frequency of the CPU to the recommended value for a given SKU in order to reduce indeterminism in the system due to frequency transitions. In the context of RAN workloads, Intel ®, in partnership with OEM manufacturers, periodically publishes the "BIOS Settings for FlexRAN™ Reference Architecture Platforms Based on Intel ® Xeon ® Processors Reference Guide". This BIOS guide is available for download from Intel Resource and Documentation Center in web based form as Document ID 861864, or as a standalone document as Document ID 861549.

Progressing to the Linux based Operating System, the system should be running a realtime kernel that supports preemption, specifically the PREEMPT_RT feature. The OS should be tuned such that the set of cores that are to be reserved for the workload application are isolated from the "housekeeping" related tasks of the kernel. The process of isolating cores varies between specific Operating System Vendors (OSVs), however in general the grub commandline is usually modified to specify a set of cores that should be placed in the isolcpus list. In addition, the same set of cores are typically placed in the rcu_nocbs along with the nohz_full list. Respectively these settings remove the set of cores from the list of candidates that will perform Read-Copy-Update (RCU) callback operations, as well as excluding the set of cores from unbound timer callback executions. Furthermore, the irqaffinity setting is used to affine IRQs to the

set of cores that will be dedicated to kernel related tasks. A similar set of tuning may also be applied through the use of services such as tuned that might layer their own workload profile onto the subset of cores that will be dedicated to running the given workload.

With regards to the Container-as-a-Service layer, the CaaS is typically based on a Kubernetes distribution and orchestrates the means by which the aforementioned tunings are applied along with automating the process of deploying the workload. Similar to the OS, the process can vary from between vendors, though in general a means is provided to generate and apply a workload profile to specify the tunings to be applied to the set of threads dedicated to the application in contrast to the kernel threads. In addition, the orchestrator usually provides a means to label a set of nodes as being suitable for realtime applications, which are then dedicated, for example, for the deployment of RAN workloads. In the context of Telco applications, the ability to separate nodes into groups based upon such labels allows Communication Service Providers and Mobile Network Operators to readily deploy workloads in a centralized fashion at scale based on the specific role, for example control or data plane, and tuning requirements, for example 5G core or 5G RAN.

Lastly, with regards to the containerized workload, this refers to the application provided by the Independent Software Vendor (ISV) that is ultimately deployed on a given CaaS platform. Although the containerized workload and the associated requirements vary between vendors, proxy workloads, such as FlexRAN™, can be used to approximate the performance of such latency sensitive applications. For example, with the Intel ® FlexRAN™ reference application, a range of lightweight and more stressful test cases over a range of use cases, such as narrow band and massive MIMO (mMIMO), can be used to understand the corresponding range of performance of the underlying system. As previously mentioned, the key considerations in this case include the ability for the mission critical application to consistently meet TTI intervals, as well as the percentage of time spent on various tasks within a given TTI interval.

Note

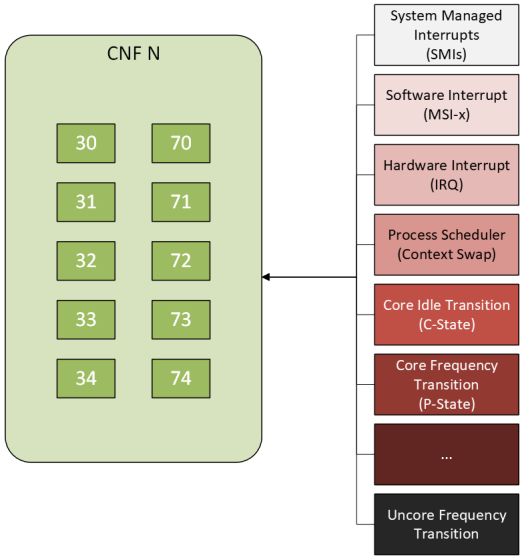
While the following guide focuses on vRAN as an example use case, the methodology provided should in general apply more broadly to other workloads. However, where applicable the specific tunings and measurement tools should be adjusted as needed for the actual workload under test. For example, there may be cases where not all of the suggested tools are required for a given workload. Similarly, there may be cases where workloads have less stringent latency requirements and so a higher maximum latency can be tolerated. Other workloads may not use as many CPU cores as what is detailed within the sample benchmarks, and so a lower number of cores can be assigned to a given measurement tool.

CHAPTER

SEVEN

BACKGROUND INFORMATION

The following figure presents an overview of the various sources of noise that have the potential to perturb and add latency to the application cores assigned to a given CNF workload. Although this list is not comprehensive, it serves as an illustrative example of the types of noise sources that should be reduced when tuning a system for latency sensitive workloads.



One example of a noise source is an interrupt, which can be in the form of a software or hardware interrupt. One such class of an interrupt is a System Management Interrupt (SMI). An SMI is typically generated by the underlying hardware platform, for example proactively performing error correction of the memory DIMMs installed within the system. When an SMI event occurs it can interrupt all of the cores within the system and generate latencies on of the order of hundreds of microseconds. Where possible SMI events should be avoided and appropriately disabled within the BIOS settings of the platform. Another class of an interrupt is a hardware Interrupt Request (IRQ). An IRQ is typically generated by a hardware component installed in the system, for example by an integrated LAN on Motherboard (LOM) controller whenever a management network packet arrives. In general, IRQs should be mapped to a subset of cores outside the list of cores assigned to the user space application. In a similar fashion, software interrupts, for example Message-Signaled Interrupts-Extended (MSI-X), where applicable should be assigned to a reserved set of cores outside the range of those assigned to the target CNF.

Another example of a noise source is a context swap, which is typically generated by the kernel process scheduler. In a multi-task operating system a given process is assigned to one or more cores. When the task completes the cores are reassigned to the next process to be executed. However, if the task does not complete within an allotted window of time, or the task is deemed to be idle, for example awaiting the completion of an I/O transaction, the kernel process scheduler can temporarily reassign the cores to another task in the queue. This core reassignment results in a context swap, which can be costly as it involves saving the current set of register values for the running process so that they

may be restored at a later point. To try and avoid such context swaps, user space application cores are typically affined to a given set of cores that will run a single workload rather than multiple workloads. Furthermore, kernel threads are typically affined to a separate set of cores to avoid kernel processes being scheduled on such user space application resources. The same principles apply to other hardware resources assigned to a given workload to help ensure isolation, for example memory regions or even cache boundaries.

A third example of a source of noise in the system is a core idle transition, or C-state transition. A C-state transition is useful for the purposes of reducing power consumption particularly during more idle periods throughout a given 24 hour period. As an example, a highly utilized core may reside in C-states C0 or C1, indicating that the core is actively executing instructions. Conversely, a core with lower utilization may reside in C-state C6 to reduce power the overall power draw of the CPU, indicating that the core is not actively executing instructions. An idle state transition has an associated latency, with the latency having a given degree of variability. Although C-state transition times have been reduced with newer CPU generations, this indeterminate latency has the potential to introduce further levels of noise, particularly for cores assigned to a given workload. In general, Intel ® Xeon ® CPUs permit the allowable set of C-states to be configured on a per-core basis. For example, if desired, administrators can disable C-state C6 exclusively for the the cores assigned to a given application of interest, while having all available C-states enabled for the cores running kernel processes.

A fourth example of a noise source is a frequency transition, either in terms of a P-state transition or an uncore frequency transition. Similar to C-state transitions, a P-state, or equivalently a frequency bin, transition is useful for the purposes of reducing power consumption during more idle periods over the course of a 24 hour period. In addition, P-states can be used on a per core basis to opportunistically increase the frequency on a given set of cores, which in turn has the potential to increase the performance of the processes executing on the corresponding cores. Similar to C-state transitions, P-states transitions introduce a level of indeterminacy on the corresponding set of cores. Uncore frequency transitions are analogous to P-state transitions, except that the uncore acts at the level of a NUMA node rather than on a per core basis. In particular uncore frequency transitions can increase or decrease the rate at which the processor die performs I/O transactions with DIMMs accessible from a given memory controller, ethernet adapters accessible from a PCIe interface, and so forth. In general, Intel ® Xeon ® CPUs permit the allowable range of P-states to be configured on a per-core basis. Similarly, the uncore frequency can be configured on a per NUMA node basis within an allowable range of frequencies. For example, if desired, administrators can lock the frequency for a given set of cores assigned to a given application of interest, along with the frequency of the uncore. In contrast to C-states, the actual running frequency for the cores as well as the uncore is limited by a set of factors, such as the maximum rated Thermal Design Power (TDP) of a given CPU. As an example, if a given set of cores requests a set of P-states that would exceed the overall TDP of the CPU, the frequency will be throttled to ensure that the power consumed by the CPU does not exceed the rated TDP. Frequent P-state transitions introduces a level of jitter into the system, in which case the core and the uncore frequency are typically locked to a fixed value for latency sensitive workloads. Since latency sensitive workloads such as RAN applications make frequent use of the AVX512 instruction set, the core frequency is locked to the a pre-determined frequency that will avoid exceeding the TDP of a given CPU SKU. The uncore frequency is also locked to a pre-determined frequency that will similarly avoid exceeding the rated TDP of the CPU.

Although not explicitly shown, other containerized or virtualized workloads collocated on the same platform might also introduce a source of noise. The challenge of managing a myriad of workloads within the same multi-tenant environment is oftentimes referred to as the “Noisy Neighbor” problem. In this case, other CNFs and/or VNFs deployed onto the system, in particular those that are not isolated from other NFs, have the potential to impact any of the shared resources on the platform. For example, a CNF might generate a context swap for the cores assigned to another CNF, consume memory or storage resources from the pool, thrash the L1 or L2 cache, or consume all of the allotted bandwidth for a network interface. Any of these conditions can result in an unexpected latency spike for a mission critical CNF. In this case, administrators typically ensure that tenant isolation is in place so that each latency sensitive workload has its own set of dedicated resources that are not shared with other CNFs and/or VNFs deployed on the platform. In addition, limits might be placed on tenants in terms of the amount of allocated cores, memory, hugepages, network interfaces, and storage. This helps to reduce the impact of the noisy neighbor scenario.

The following figure presents a sample TTI budget allocation for a RAN application. In this example, a given 1 millisecond TTI interval is subdivided into time spent on RAN application related processing along with time spent on system related tasks. In an ideal scenario, RAN application processing time would account for 100% of the TTI interval, and the system related tasks would account for 0% of the TTI interval. The optimization goal is to reduce the system pro-

cessing overhead to as close to 0% as is realistically possible. As shown in the diagram, the application is subdivided further into the typical set of tasks that would be carried out in a RAN scenario, such as DownLink (DL) processing, UpLink (UL) processing, Sounding Reference Signal (SRS) processing, along with Forward Error Correction (FEC) processing.

$$t_{RAN} = t_{DL} + t_{UL} + t_{SRS} + t_{FEC} \tag{7.1}$$

Refer to the following table for the definitions of each term within this relation.

Term	Definition
t_{RAN}	The amount of time spent by the RAN application
t_{DL}	The DL processing latency
t_{UL}	The UL processing latency
t_{SRS}	The SRS processing latency
t_{FEC}	The aggregate FEC processing latency accounting for both UL and DL

In addition, the system overhead collectively represents the amount of time spent handling context swaps, SMIs, or any of the other potential sources of noise mentioned above.

$$t_{system} = t_{SMI} + t_{NMI} + t_{MSI} + t_{IRQ} + t_{CS} + t_{IT} + t_{FT} + t_{UFT} + t_{NN} \tag{7.2}$$

Refer to the following table for the definitions of each term within this relation.

Term	Definition
t_{system}	The amount of time spent on system overhead
t_{SMI}	The SMI processing latency
t_{NMI}	The NMI processing latency
t_{MSI}	The MSI-x processing latency
t_{IRQ}	The IRQ processing latency
t_{CS}	The context switch processing latency
t_{IT}	The idle transition processing latency
t_{FT}	The core frequency transition processing latency
t_{UFT}	The uncore frequency transition processing latency
t_{NN}	The latency introduced by all remaining noisy neighbor workloads

In order to safely avoid a TTI violation, at every instance of time the following relationship must hold true:

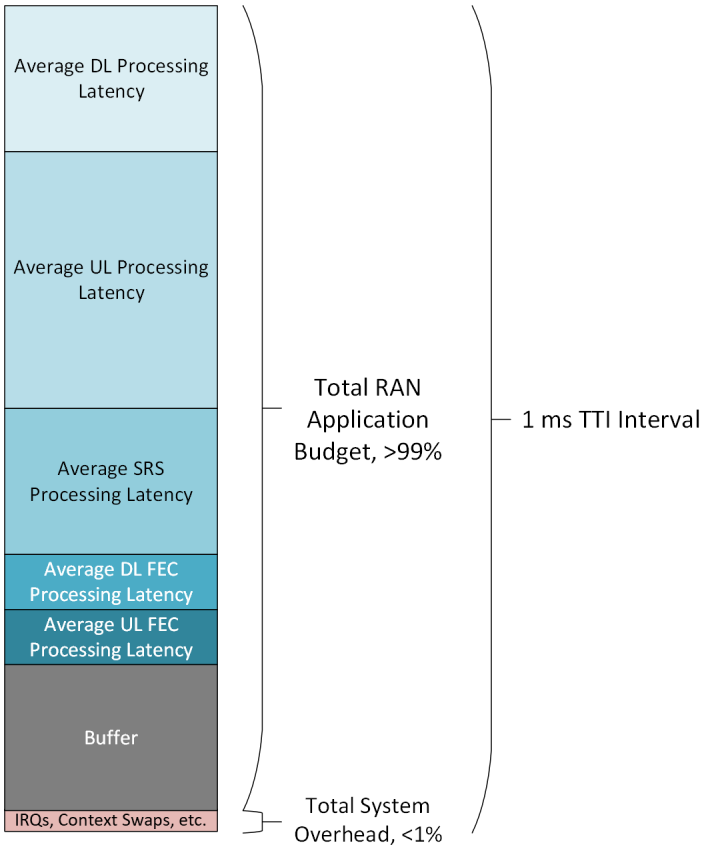
$$t_{RAN} + t_{system} < TTI \tag{7.3}$$

In particular, any time that is left over and not consumed either by the RAN application or the system is referred to as the buffer for a given TTI interval:

$$t_{buffer} = TTI - (t_{RAN} + t_{system}) \tag{7.4}$$

Where t_{buffer} represents the amount of buffer time remaining within a given TTI interval.

Given that values above all represent instantaneous values that change from one TTI interval to the next, this variability implies that more or less time can be spent on any of the RAN or system related tasks mentioned above. Given this inherent variability, the goal of minimizing the latency due to system overhead also achieves the equally important goal of maximizing the amount of buffer for each TTI interval. The lower the system overhead, the greater the buffer, and in turn the greater the probability of avoiding a TTI violation.


Average DL Processing Latency
Average UL Processing Latency
Average SRS Processing Latency
Average DL FEC Processing Latency
Average UL FEC Processing Latency
Buffer
IRQs, Context Swaps, etc.
Total RAN Application Budget, >99%
Total System Overhead, <1%
1 ms TTI Interval

Note

- The previous diagram is not rendered to scale and is primarily for illustrative purposes only.
- The latencies shown are instantaneous and do not include error bars that would indicate their standard deviations over multiple TTI intervals.
- The latencies shown do not represent an exhaustive list of tasks that may be carried out in a given TTI interval.

CHAPTER

EIGHT

KPI DEFINITIONS

8.1 Abstract KPIs

Abstract KPIs are not bound to specific measurement tools and represent general latency criteria applicable across different measurement approaches.

Maximum Value
Maximal latency spike duration.

Pass or Fail
Binary assessment indicating whether the maximum value exceeds latency threshold.

Mean Time Between Failures (MTBF)
Statistical measure of the average operational time between latency spikes exceeding latency thresholds.

Rough approximation:

$$MTBF = \frac{Total\ Operating\ Time}{Number\ of\ Failures}$$

CPU Availability
The percentage of runtime that was free from noise related latencies.

Aggregate Percentage
The percentage of samples at or below a specified latency threshold across all threads.

$$Aggregate\ Percentage = \frac{\sum_{i=1}^{n} Samples_{\leq threshold,i}}{\sum_{i=1}^{n} Total_Samples_i} \times 100\%$$

where n is the number of threads, $Samples_{\leq threshold,i}$ is the count of samples at or below the threshold for thread i, and $Total_Samples_i$ is the total number of samples for thread i.

This section presents details regarding the specific metrics that are to be measured along with definitions for each of the measurement tools of interest. For example, the average, and maximum latency reported by cyclictest, the maximum noise reported by hwnoise, and so forth. This list of metrics distinguishes between the values reported directly by each of the corresponding tools, which are labeled as "raw" metrics, along with metrics that are calculated based upon these raw metrics, which are labeled as "derived" metrics. An example of a raw metric would be the maximum latency reported by cyclictest. An example of a derived metric would be the percentage of samples that are at or below a latency threshold of 10 μs, which are calculated based on the histogram generated by cyclictest. Where applicable, units are also included with the associated measurement. Several metrics are not KPIs because they depend on test duration. The following table provides the list of metrics that will be of interest within this document.

Metric	KPI	Definition	Measurement Tool	Derived or Raw Measurement	Units
Maximum Latency	KPI	The maximum latency measured for a given core	cyclictest	Raw	μs
Total Count above Threshold		The count of the number of recorded latency values above 10 μs	cyclictest	Derived	N/A
Aggregate Percentage at or below 10 μs	KPI	The count of the number of recorded latency values at or below 10 μs expressed as a percentage	cyclictest	Derived	%
Max Latency	KPI	The maximum latency observed during the given runtime	hwlatdetect	Raw	μs
Samples exceeding threshold		The number of samples exceeding the target latency threshold	hwlatdetect	Raw	Count
Maximum	KPI	The maximum latency observed during a busy loop	oslat	Raw	μs
IRQ Timer Latency avg		The average IRQ timer latency	timerlat	Raw	μs
IRQ Timer Latency max		The maximum IRQ timer latency	timerlat	Raw	μs
Thread Timer Latency avg		The average thread timer latency	timerlat	Raw	μs
Thread Timer Latency max		The maximum thread timer latency	timerlat	Raw	μs
Ret Timer Latency avg		The average timer latency for a user-space thread encompassing both kernel and user-space processing time for the given task	timerlat	Raw	μs
Ret Timer Latency max	KPI	The maximum timer latency for a user-space thread encompassing both kernel and user-space processing time for the given task	timerlat	Raw	μs
Noise		The sum of all the noise latencies observed during the given runtime	hwnoise	Raw	μs
CPU Availability	KPI	The percentage of runtime that was free from noise related latencies	hwnoise	Raw	%
Max Noise		The maximum amount of noise latency within a given period interval	hwnoise	Raw	μs
Max Single	KPI	The maximum latency introduced by a single noise event	hwnoise	Raw	μs
HW		The number of hardware related noise events during the given runtime	hwnoise	Raw	Count
NMI		The number of Non-Maskable Interrupts (NMIs) during the given runtime	hwnoise	Raw	Count

CHAPTER

NINE

MEASUREMENT TOOLS

This section provides details for the specific measurement tools of interest, for example cyclictest, oslat, hwlatdetect, hwnoise, and so forth, that are used to help quantify whether a system has been tuned for latency sensitive workloads. This list is intended to be comprehensive enough to provide a holistic picture of the amount of "noise", or lack thereof, within a given system, and in turn to assess the level of compliance for a given system to run such latency sensitive workloads, such as RAN, in a deterministic fashion. Each of the tool descriptions also includes a relevant link for interested readers to learn more. This study includes the following measurement tools:

hwlatdetect
: Designed to uncover latency issues caused by the underlying hardware or firmware, rather than the operating system itself. It temporarily suppresses most software activity to isolate and detect delays introduced by things like System Management Interrupts (SMIs), making it valuable for diagnosing platform-level latency issues.

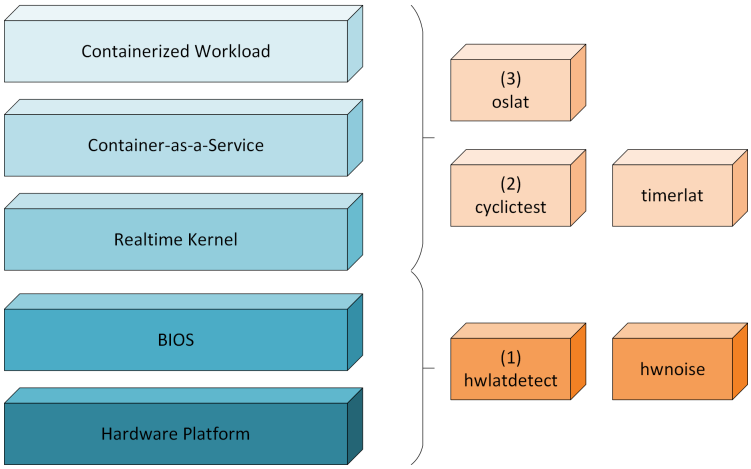
hwnoise
: Focuses on detecting unpredictable latency caused by hardware-level noise, such as firmware activities. It monitors for irregular timing behavior that can disrupt real-time operations, helping to identify and mitigate sources of jitter that are otherwise hard to trace.

cyclictest
: A widely used tool for measuring the wakeup latency of the Linux kernel's scheduler. It works by entering high-resolution sleeps and recording how long it takes for cyclictest to wake up once those sleeps expire. This helps evaluate how reliably and quickly the system can handle real-time tasks.

oslat
: Measures gaps in the high-resolution timer under optional load by running multiple threads pinned to specific CPUs and timing their responses to scheduled events. It's particularly useful for understanding how latency behaves in multi-core environments, especially when the system is under stress or heavy workload.

timerlat
: Provides fine-grained insights into timer-related latency within the Linux kernel. By tracing the time between when a timer expires and when its handler actually runs, it helps developers and system integrators pinpoint delays that could affect real-time performance.

The following figure presents an overview of where each of tools mentioned above lies within the overall system under study. For example, the hwnoise and hwlatdetect tools are primarily targeted for measuring the latency and jitter introduced by the underlying hardware platform along with the relevant BIOS settings. By contrast, cyclictest, oslat, and timerlat are primarily targeted for measuring the latency and jitter introduced by the kernel, the CaaS layer, along with other containerized workloads collocated within the multi-tenant environment. Included within this diagram is the recommended order in which each tool should be run. Following the layered approach, tools such as hwlatdetect should be run first in order to identify any spikes related to hardware events, while tools such as oslat should be run last after the user has gained confidence that the lower layers are sufficiently tuned.


Containerized Workload
Container-as-a-Service
Realtime Kernel
BIOS
Hardware Platform
(3)
oslat
(2)
cyclictest
timerlat
(1)
hwlatdetect
hwnoise

CHAPTER

TEN

INITIAL SYSTEM SETUP

This section provides a general overview of the preliminary setup process of a given platform for running latency sensitive workloads. The setup process includes a sample hardware BOM, BIOS configuration, a sample software BOM, a sample CaaS deployment, and a sample YAML template for a performance profile along with the associated grub command line. Afterwards, the system should be at a state where each of the previously described measurement tools can be executed on the platform.

10.1 Hardware BOM

The following table provides a sample Bill of Materials for a hardware platform suitable for deploying and benchmarking latency sensitive workloads.

Hardware	Description
Processors	Intel ® Xeon ® 6 SoC with Intel ® vRAN Boost 2.0
Processor SKU	Intel ® Xeon ® 6 SoC 6706P-B 40C80T 235W
DRAM	32GB 2Rx8 6400 MT/s populated 1 DIMM per channel for a total of 128 GB available memory
Integrated Network Interface Card	Intel ® Ethernet Network Adapter E825XXV-DA8 (Gen4 x8)
Discrete Network Interface Card	Intel ® Ethernet Network Adapter E830XXV-DA8 OCP 3.0
Storage	M.2 NVMe solution as boot device

10.2 BIOS Configuration

The following describes the BIOS configuration settings required to achieve optimal performance for latency sensitive workloads such as RAN applications. Typically OEM vendors provide a workload profile specific to RAN, such as a "Telco Optimized Profile" or a "vRAN" specific profile. These profiles are designed to meet optimized performance for such RAN workloads. In the event that such a profile is unavailable, Intel provides a set of recommended BIOS settings for maximum performance for low latency configurations to meet the optimized deterministic performance requirements for RAN. Refer to the document "BIOS Settings for FlexRAN™ Platforms based on Intel ® Xeon ® Processors" (Document ID #861864) for information on the relevant settings. The document is available for download from: Document ID #861864. This document provides details for multiple generations of Intel ® Xeon ® based processors. With reference to the given hardware platform please reference Section 3.5 "Intel ® Xeon ® 6 SoC".

Note

The following guide is updated on a regular basis. Be sure to download the latest version for the most up to date guidance regarding the recommended BIOS settings. Please work with an appropriate representative from Intel in the event this document is not accessible.

10.3 Software BOM

The following table provides a sample Bill of Materials for a software stack suitable for deploying and benchmarking latency sensitive workloads. Note as before the use of a realtime operating system for such timing sensitive applications.

Ingredient	SW Version Details
DPDK	24.11
FlexRAN™	25.07
Red Hat OpenShift Container Platform	4.20
ice	5.14.0-570.60.1.el9_6.x86_64+rt
iavf	5.14.0-570.60.1.el9_6.x86_64+rt
E825 NVM	4.03
E830 NVM	1.11
Real-time Kernel	5.14.0-570.60.1.el9_6.x86_64+rt
DDP	ICE OS Default Package version 1.3.43.0

Note

The following table represents the recommended versions as of the date of the publication for this guide. Only the latest versions, including the BIOS and microcode versions, should be used when benchmarking a given system.

10.4 CaaS Deployment

A sample Container-as-a-Service platform would be the Red Hat OpenShift Container Platform (RHOCP). For instructions on how to install RHOCP with Zero Touch Provisioning (ZTP) please refer to the Edge computing documentation for Red Hat OpenShift 4.20.

10.5 Performance Profile

The following presents a sample Performance Profile for the Node Tuning Operator included with the Red Hat OpenShift Container Platform. This example assumes the use of a 40C80T Intel ® Xeon ® 6 SoC 6706P-B CPU.

```
apiVersion: performance.openshift.io/v2
kind: PerformanceProfile
metadata:
  name: openshift-node-performance-profile
  annotations:
    ran.openshift.io/ztp-deploy-wave: "10"
    ran.openshift.io/reference-configuration: "ran-du.redhat.com"
spec:
  additionalKernelArgs:
    - rcupdate.rcu_normal_after_boot=0
    - efi=runtime
```

(continues on next page)

(continued from previous page)

```
    - vfio_pci.enable_sriov=1
    - vfio_pci.disable_idle_d3=1
    - module_blacklist=irdma
  cpu:
    isolated: 2-41,44-83
    reserved: 0-1,42-43
  hardwareTuning:
    isolatedCpuFreq: 2500000
    reservedCpuFreq: 2500000
  hugepages:
    defaultHugepagesSize: 1G
    pages:
      - size: 1G
        count: 32
  machineConfigPoolSelector:
    pools.operator.machineconfiguration.openshift.io/$mcp: ""
  nodeSelector:
    node-role.kubernetes.io/$mcp: ""
  numa:
    topologyPolicy: restricted
  realTimeKernel:
    enabled: true
  workloadHints:
    realTime: true
    highPowerConsumption: false
    perPodPowerManagement: false
```

Note

Please adjust the core assignment and memory based on the specific hardware platform under test.

The following presents a sample of the relevant parameters within the grub commandline generated for a system populated with a 40C80T Intel x86 CPU.

Parameters	Value	Description
intel_iommu	on	Intel memory protection enabled
iommu	pt	Memory protection in passthrough mode for direct access
skew_tick	1	Spreads out timer interrupts for better performance
tsc	reliable	Uses CPU timer as reliable time source
rcup-date.rcu_normal_after_boo	0	Uses fast expedited kernel cleanup after boot
nohz	on	Tickless kernel mode saves power
rcu_nocbs	2-39,42-79	CPUs 2-39 and 42-79 skip callback processing
tuned.non_isolcpus	00000003,0000000	CPUs available for tuned service in bitmask format
systemd.cpu_affinity	0,1,40,41	System manager runs only on CPUs 0,1,40,41
isolcpus	managed_irq,2-39,42-79	CPUs 2-39,42-79 reserved with managed interrupts
nohz_full	2-39,42-79	CPUs 2-39,42-79 run without timer interrupts
nosoftlockup		Soft hang detection disabled
nmi_watchdog	0	System hang monitor disabled
mce	off	Hardware error checking disabled
rcutree.kthread_prio	11	Kernel cleanup threads run at high priority 11
default_hugepagesz	1G	Default large memory pages are 1GB each
hugepagesz	1G	Large memory pages set to 1GB size
hugepages	32	Allocates 32 large memory pages (32GB total)
intel_pstate	active	Intel CPU frequency control in active mode
efi	runtime	Firmware services enabled at runtime
vfio_pci.enable_sriov	1	Virtual device sharing enabled
vfio_pci.disable_idle_d3	1	Prevents devices from entering deep sleep
module_blacklist	irdma	Blocks irdma driver from loading
sys-temd.unified_cgroup_hiera	1	Modern process grouping enabled
cgroup_no_v1	all	All old process grouping methods disabled
psi	0	System pressure monitoring disabled
rcutree.nohz_full_patience_	1000	Set delay before RCU enters full nohz mode, balancing responsiveness and efficiency

CHAPTER

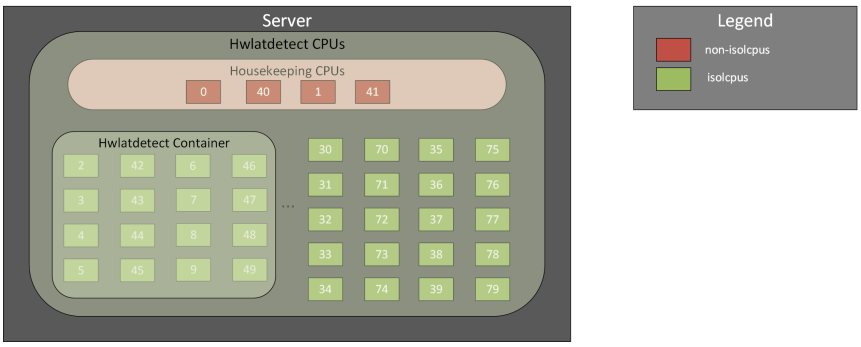
ELEVEN

MEASUREMENT TOOL SETUP

This section includes details for how each of the previously referenced measurement tools are deployed onto the platform along with guidance regarding their execution. Note that the order specified below aligns with the order in which each of the tools should be run. As previously mentioned, a layered approach should be adopted in which lower layers are certified first using tools like hwlatdetect. Only after a given test case passes should the user proceed to the next test case. If a given test case is not passing, the system should be reviewed to confirm that the appropriate tunings are in place.

11.1 Hwlatdetect

The following figure presents a diagram of the hwlatdetect workload running inside a container on the DUT. Note that the given core assignment is for example purposes only. When necessary the core assignment should be adjusted to account for the specific CPU installed in the System Under Test.



To run the hwlatdetect test case as a container on the DUT:

```
podman run -v $(pwd)/:/kubeconfig:Z \
-e KUBECONFIG=/kubeconfig/kubeconfig \
-e LATENCY_TEST_CPUS=16 \
-e LATENCY_TEST_RUNTIME=43200 \
-e MAXIMUM_LATENCY=15 \
registry.redhat.io/openshift4/cnf-tests-rhel9:v4.20 \
/usr/bin/test-run.sh --ginkgo.focus="hwlatdetect" --ginkgo.v --ginkgo.timeout="13h"
```

For the following test case:

1. The test should run for a duration of 12 hours.
2. The test should specify a latency threshold of 15 microseconds.
3. Once the test completes record the maximum latency value along with the number of samples exceeding the threshold.

Pass/Fail Criteria:

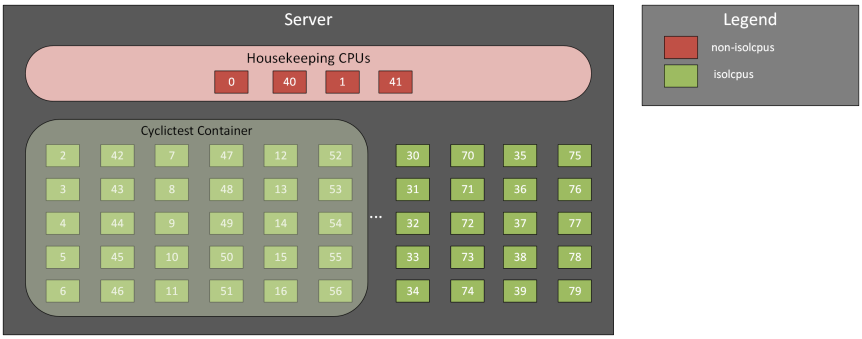
- The DUT must demonstrate a maximum latency below the specified threshold, specifically with zero sampling events above the target threshold for cores in the isolcpus list.

Note

- Although the LATENCY_TEST_CPUS is required by the hwlatdetect test case and determines where the container will run, the hwlatdetect test case will actually run on all available CPUs, including those not in the isolcpus list.
- This test leverages the cnf-tests image, which is specific to RHOCP environments.
- It is expected that the latencies can be higher on the cores in the non-isolcpus list. If the samples above the target threshold occur on cores in the non-iscolcpus list is well above the maximum then this would warrant further investigation.
- If the test case results for hwlatdetect are not meeting the specified criteria, the ancillary hwnoise test case can be run in order to debug the system further. Refer to the appendix for additional information on how to run the hwnoise test case.
- For more information on running hwlatdetect please refer to Performing latency tests for platform verification.

11.2 Cyclictest

The following figure presents a diagram of the cyclictest workload running inside a container on the DUT. Note that the given core assignment is for example purposes only. When necessary the core assignment should be adjusted to account for the specific CPU installed in the System Under Test.



To run the cyclictest test case as a container on the DUT:

```
podman run -v $(pwd)/:/kubeconfig:Z \
           --name=cyclictest \
           -e KUBECONFIG=/kubeconfig/kubeconfig \
           -e LATENCY_TEST_CPUS=30 \
           -e LATENCY_TEST_RUNTIME=43200 \
           -e LATENCY_TEST_DELAY=60 \
           registry.redhat.io/openshift4/cnf-tests-rhel9:v4.20 \
           /usr/bin/test-run.sh --ginkgo.focus="cyclictest" --ginkgo.v --ginkgo.timeout=
↪"13h"
```

For the following test case:

1. The LATENCY_TEST_CPUS specifies the list of CPUs to test with the cyclictest command. Execute cyclictest against 15 cores (30 threads) on the system.
2. The test should run for a duration of 12 hours (43,200 seconds).
3. Once the test completes the cyclictest measurement tool will generate a histogram where each bin represents a specific latency value in microseconds. Record the histogram results.
4. Based on the histogram calculate the number of samples at or below 10 microseconds expressed as a percentage.
5. Based on the histogram calculate the number of samples at or below 30 microseconds expressed as a percentage.
6. Based on the maximum latency value reported per thread, record the maximum latency across all threads.

Pass/Fail Criteria:

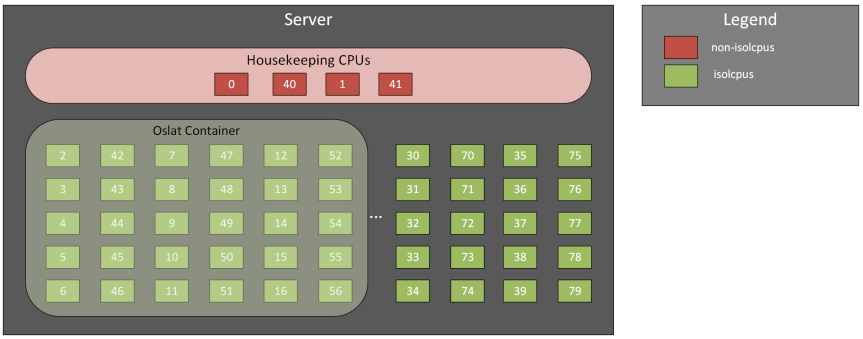
- The DUT must demonstrate the system latency for the wake-up time of the threads to be at or below 10 microseconds for an aggregate percentage of 99.999%, with C-state C0 only enabled, and a latency at or below 30 microseconds for an aggregate percentage of 99.9999%.

Note

- It is not recommended to run cyclictest against more than 15C30T on the system. Specifically it is not recommended to execute cyclictest against all available cores in the isolcpus list.
- An aggregate percentage across all threads should be used when calculating the number of samples at or below 10 microseconds rather than individual percentages across each thread.
- This test leverages the cnf-tests image, which is specific to RHOCP environments.
- The LATENCY_TEST_DELAY will wait 60 seconds before starting cyclictest within the container.
- If the test case results for cyclictest are not meeting the specified criteria, the ancillary timerlat test case can be run in order to debug the system further. Refer to the appendix for additional information on how to run the timerlat test case.
- For more information on running cyclictest please refer to Performing latency tests for platform verification.

11.3 Oslat

The following figure presents a diagram of the oslat workload running inside a container on the DUT. Note that the given core assignment is for example purposes only. When necessary the core assignment should be adjusted to account for the specific CPU installed in the System Under Test.



To run the oslat test case as a container on the DUT:

```
podman run -v $(pwd)/:/kubeconfig:Z \
-e KUBECONFIG=/kubeconfig/kubeconfig \
-e LATENCY_TEST_CPUS=30 \
-e LATENCY_TEST_RUNTIME=43200 \
-e LATENCY_TEST_DELAY=60 \
-e MAXIMUM_LATENCY=30 \
registry.redhat.io/openshift4/cnf-tests-rhel9:v4.20 \
/usr/bin/test-run.sh -ginkgo.v -ginkgo.focus="oslat" --ginkgo.timeout="13h"
```

For the following test case:

1. The LATENCY_TEST_CPUS specifies the list of CPUs to test with the oslat command. Execute oslat against 15 cores (30 threads) on the system.
2. The test should run for a duration of 12 hours (43200 seconds).
3. The test should specify a maximum latency (MAXIMUM_LATENCY) of 20 microseconds.
4. Based on the histogram calculate the number of samples at or below 10 microseconds expressed as a percentage.
5. Based on the histogram calculate the number of samples at or below 30 microseconds expressed as a percentage.
6. Based on the maximum latency value reported per thread, record the maximum latency across all threads.

Pass/Fail Criteria:

- The DUT must demonstrate a latency across all threads at or below 10 microseconds for an aggregate percentage of 99.999% and a latency at or below 30 microseconds for an aggregate percentage of 99.9999%.

Note

- Similar to cyclictest, it is not recommended to run oslat against more than 15C30T on the system. Specifically it is not recommended to execute oslat against all available cores in the isolcpus list.
- The LATENCY_TEST_DELAY will wait 60 seconds before starting cyclictest within the container.
- This test leverages the cnf-tests image, which is specific to RHOCP environments.
- For more information on running oslat please refer to Performing latency tests for platform verification.

CHAPTER

TWELVE

SAMPLE BENCHMARK RESULTS

The following section includes sample benchmark data for cyclictest, oslat, hwlatdetect, hwnoise, and timerlat. For each of the associated measurement tools, both a positive and a negative example are provided for illustrative purposes. A system to be deemed compliant for latency sensitive workloads should be able to meet or exceed all of the following results.

Note

- All of the examples shown below are for illustrative purposes only.
- For specific examples of benchmark data for an existing combination of hardware platform, CPU, OS, and CaaS variant, where available to be shared under a Non-Disclosure Agreement (NDA), please reach out to your Intel representative.

12.1 Hwlatdetect

The following presents a positive example for a hwlatdetect benchmark result.

Latency Threshold	15 μs
Sample Window	1000000 μs
Sample Width	950000 μs
Non-sampling Period	50000 μs
Max Latency	Below threshold
Samples exceeding threshold	0

The following presents a negative example for a hwlatdetect benchmark result.

Latency Threshold	15 μs
Sample Window	1000000 μs
Sample Width	950000 μs
Non-sampling Period	50000 μs
Max Latency	1025 μs
Samples exceeding threshold	1284

12.2 Hwnoise

The following presents an example for a hwnoise benchmark result.

CPU	Period	Runtime	Noise	% CPU Aval	Max Noise	Max Single	HW	NMI
2	#3600	2.70E+09	184	99.99999	10	9	64	0
3	#3600	2.70E+09	140	99.99999	10	9	46	0
4	#3600	2.70E+09	136	99.99999	11	9	49	0
5	#3600	2.70E+09	103	99.99999	8	7	43	0
6	#3600	2.70E+09	118	99.99999	8	7	44	0
34	#3600	2.70E+09	182	99.99999	11	9	61	0
35	#3600	2.70E+09	89	99.99999	10	9	36	0
36	#3600	2.70E+09	134	99.99999	9	9	47	0
37	#3600	2.70E+09	80	99.99999	11	9	30	0
38	#3600	2.70E+09	67	99.99999	10	9	29	0

12.3 Cyclictest

The following presents a positive example for a cyclictest benchmark result.

Minimum Count Per Core at Median Latency	1241678759
Minimum Latency (μs)	2
Maximum Latency (μs)	7
Total (Count)	54321222
Total above 10 μs (Count)	0
Total at or above 10 μs (%)	0%
Total at or below 10 μs (%)	100%

The following presents a negative example for a cyclictest benchmark result.

Minimum Count Per Core at Median Latency	669783368
Minimum Latency (μs)	2
Maximum Latency (μs)	306
Total (Count)	1266940469
Total above 10 μs (Count)	21204
Total at or above 10 μs (%)	0.001636111%
Total at or below 10 μs (%)	99.998363889%

12.4 Oslat

The following presents a positive example for an oslat benchmark result.

core_i:	2	...	8	9
cpu_mhz:	2197	...	2197	2197
001 (μs):	93068541026	...	93068541320	93068541409
002 (μs):	51	...	51	51
003 (μs):	2	...	2	2
004 (μs):	4	...	4	2

continues on next page

Table 1 – continued from previous page

005 (μs):	40	...	40	41
006 (μs):	1	...	1	1
007 (μs):	0	...	0	0
008 (μs):	0	...	0	0
009 (μs):	0	...	0	0
010 (μs):	0	...	0	0
011 (μs):	0	...	0	0
012 (μs):	0	...	0	0
013 (μs):	0	...	0	0
014 (μs):	0	...	0	0
015 (μs):	0	...	0	0
016 (μs):	0	...	0	0
017 (μs):	0	...	0	0
018 (μs):	0	...	0	0
019 (μs):	0	...	0	0
020 (μs):	0	...	0	0
021 (μs):	0	...	0	0
022 (μs):	0	...	0	0
023 (μs):	0	...	0	0
024 (μs):	0	...	0	0
025 (μs):	0	...	0	0
026 (μs):	0	...	0	0
027 (μs):	0	...	0	0
028 (μs):	0	...	0	0
029 (μs):	0	...	0	0
030 (μs):	0	...	0	0
031 (μs):	0	...	0	0
032 (μs):	0	...	0	0
maxlat (μs):	6	...	6	6
runtime (sec):	3600.740	...	3600.740	3600.740

The following presents a negative example for an oslat benchmark result.

```
oslat:
[FAILED] The maximum latency measured 22 is bigger than the expected one 20
Expected
    <bool>: false
to be true
```

12.5 Timerlat

The following presents an example for a timerlat benchmark result.

```
                |          IRQ Timer Latency (us)        |         Thread Timer Latency (us)
CPU COUNT       |      cur       min       avg       max |      cur       min       avg     ␣
↪  max
0   58634       |        1         0         1        10 |       11         2        10     ␣
↪   23
1   58634       |        1         0         1         9 |       12         2         9     ␣
↪   23
2   58634       |        0         0         1        11 |       10         2         9     ␣
```

(continues on next page)

(continued from previous page)

```
↪  23
3   58634    |        1        0        1        11 |        11        2        9
↪  24
4   58634    |        1        0        1        10 |        11        2        9
↪  26
5   58634    |        1        0        1         8 |        10        2        9
↪  25
6   58634    |        1        0        1         9 |        11        2        9
↪  23
7   58633    |        1        0        1         9 |        11        2        9
↪  26
8   58633    |        1        0        1         9 |        10        2        9
↪  26
9   58633    |        1        0        1        13 |        11        2        9
↪  28
10  58633    |        1        0        1        13 |        12        2        9
↪  24
11  58633    |        1        0        1         8 |        10        2        9
↪  23
12  58633    |        1        0        1        10 |        10        2        9
↪  22
13  58633    |        1        0        1        18 |        12        2        9
↪  27
14  58633    |        1        0        1        10 |        11        2        9
↪  28
15  58633    |        0        0        1        11 |         7        2        9
↪  26
16  58633    |        1        0        1        13 |        10        2        9
↪  24
17  58633    |        1        0        1         9 |        13        2        9
↪  22
18  58633    |        1        0        1        10 |        11        2        9
↪  23
19  58633    |        1        0        1        12 |        11        2        9
↪  28
20  58633    |        1        0        1        14 |        11        2        9
↪  24
21  58633    |        1        0        1         8 |        11        2        9
↪  22
```

CHAPTER

THIRTEEN

CONCLUSION

To summarize, some of the key concepts that were discussed within this guide include the following:

- In general Latency sensitive applications are categorized as mission critical workloads that must meet strict timing requirements. Such workloads cover multiple domains and use cases including industrial IoT, financial sector trading, as well as Radio Access Networks.
- RAN applications in particular are required to continually meet their Transmission Time Interval (TTI) windows. A TTI violation implies that the application was not able to meet the necessary timing requirement within the allowed window.
- Part of the optimization goal of tuning a system for latency sensitive workloads is to reduce the amount of time spent on system related tasks, or conversely to maximize the amount of buffer for a given TTI interval.
- In order to ensure that the majority of the TTI budget is devoted to RAN related processing it is necessary to reduce, to the greatest extent possible, any source of jitter or noise that could potentially interrupt resources that are allocated to latency sensitive workloads. Such hardware resources can include CPU cores, memory, cache, network interfaces, among others.
- For CPU cores in particular, sources of noise can include context switches, frequency transitions, idle transitions, SMIs, IRQs, among others.
- Several tools can be used to measure the amount of jitter and noise that a system is subjected to. For example, hwlatdetect can be used to measure latency spikes at a platform level, such as those due to an SMI event. As another example, cyclictest can be used to measure latency spikes on a per-core level.
- Such measurement tools can help identify and root cause such sources of noise, for example unexpected context swaps due to a noisy neighbor workload, or an SMI event due to a platform related task.
- These measurement tools should be run in a systematic manner in a layered approach starting with the hardware layer and gradually progressing up the stack to the CaaS and concluding with the application layer. Only when a given layer passes the necessary latency requirements should the next layer be benchmarked. In addition, it is recommended to run each benchmark on a subset of cores, rather than on every available core within the system.
- Collectively, the presented benchmark methodology and sample results can be used to certify a system as compliant for running latency sensitive workloads. This in turn increases the probability that a given application can consistently meet the specified TTI intervals.
- Tuning a system for latency sensitive workloads involves a holistic approach, where the system is subdivided into layers, and the appropriate set of optimizations are applied to each respective layer. For example, for RAN applications the hardware platform must be populated with memory in a 1 DPC fashion, the BIOS should set to the appropriate workload profile, the OS should support a real-time kernel and have the appropriately set grub commandline parameters, and the CaaS should support workload isolation in an orchestrated manner.

For future revisions of the guide:

- The current PDF format will be converted into web document format to help broaden the intended distribution and further accelerate the release of newer revisions.
- The list of example operating systems and CaaS offerings will be expanded to include additional OS and CaaS variants.
- The benchmark methodology will be expanded to include more concrete examples for specific OS and CaaS variants.

CHAPTER

FOURTEEN

REFERENCES

The following section provides references to industry standards that should be adhered to in the context of deploying latency sensitive workloads, for example meeting the stringent Transmission Time Intervals (TTI) for RAN applications, along with performing Precision Time Protocol and Synchronous Ethernet synchronization.

14.1 Timing and Synchronization Standards

The IEEE 1588-2008 and IEEE 1588-2019 guides document the Precision Time Protocol (PTP) version 2 and version 2.1 standards and provide the foundation of precise packet-based timing synchronization. They define the PTP standard including clock roles (Grandmaster, Boundary, and Transparent clocks) and are required for the timing alignment of Radio Access Network (RAN) applications, for example MMIMO deployment. In the context of RAN these standards enable sub-microsecond synchronization accuracy between distribute radio units (RUs), distributed units (DUs), and centralized units (CUs). The reference profiles in this case include ITU-T G.8275.1, which provides a full timing support implementation, an ITU-T G.8275.2, which provides partial timing support.

In addition, the ITU-T G.8261, G8262, G8264 standards document the requirements for Synchronous Ethernet (SyncE). The G.8261 standard specifies the overall architecture and performance limits for packet-based frequency synchronization. Furthermore, the G.8262 standard defines the Ethernet Equipment Clock (EEC) requirements in terms of wander, jitter, and holdover. Moreover, the G.8264 specification defines the Ethernet Synchronization Messaging Channel (ESMC) standard. In the context of RAN deployments, SyncE is used in conjunction with PTP to provide frequency stability during PTP holdover or in the event of link failures. Typical performance targets for SyncE include frequency accuracy on the order of sub 16 parts per billion (ppb) at air interface reference.

In terms of phase and timing synchronization the ITU-T G.8271, G.8272, G.8273, and G.8275 standards cover the key latency and phase error control specifications for RAN. The G.8271 standard discusses on the timing and phase synchronization architecture along with performance. In this case, the key target requirement is an end-to-end network limit of +/- 1.5 microseconds for 5G Time Division Duplex (TDD) relative time error. The G.8272 standard focuses on the Primary Reference Time Clock (PRTC), with a target of 100 nanoseconds (1 PPS) traceable to the UTC. The G.8273.2 standard addresses boundary and transparent clock performance, while the G.8275 standard focuses on Network PTP profiles and defines how the delay, asymmetry, and packet delay variation are to be handled. Collectively these standards define the absolute versus relative time error budgets and specify how timing packets must traverse the network in order to maintain accuracy.

14.2 Transport and Latency Budget Standards

The IEEE 802.1AS-2020 (gPTP) standard documents the Time-Sensitive Networking (TSN) adaptation of PTP for Ethernet protocols, which is relevant whenever RAN elements share infrastructure within TSN domains, and maintains bounded latencies for deterministic time propagation, for example less than 1 microsecond. The IEEE 1914.1 and 1914.3 packet-based front-haul defines the latency, jitter, and synchronization requirements for Centralized RAN (C-RAN) and front-haul over Ethernet. Some of the key requirements include a round-trip latency less than or equal to 250 microseconds, a Packet Delay Variation (PDV) of less than 100 nanoseconds for synchronization traffic, and jitter less than 10 nanoseconds for timing signals. The 3GPP specifications for timing and TTI alignments are documented within

3GPP TS 38.104, TS 38.133, TS 38.211, TS 38.213, and TR 38.801. The 3GPP TS 38.104 specification describes the base station (gNB) radio performance requirements, such as the frame timing accuracy, typically +/- 1.5 microseconds cell-to-cell for Time Division Duplex (TDD) RAN. Furthermore, 3GPP TS 38.133 describes the requirements for synchronization accuracy between Radio Units (RUs), which must maintain a phase alignment, for example less than or equal to 260 nanoseconds for a 5G New Radio (5GNR) TDD sub-carrier spacing greater than or equal to 30 kHz. The 3GPP TS 38.211 and TS 38.213 specifications describe the physical channels and timing structures, for example with definitions of Transmission Time Intervals (TTIs) of 1 ms, 0.5 ms, or 0.25 ms depending on the use case. In addition, the 3GPP TR 38.801 specification describes the study of NR RAN architectures and contains timing distribution models along with methods for propagation delay compensations.

14.3 Best Practice and Industry Implementation Guidance

The Alliance for Telecommunications Industry Solutions (ATIS) provides practical end-to-end design examples for deploying solutions that include PTP, SyncE, along with GNSS with holdover. In addition, the Open RAN (O-RAN) Working Groups (WG), for example WG4, provide definitions for synchronization models and acceptable latency for RU-DU interconnects, for example a 100 nanosecond differential delay. Furthermore, the Small Cell Forum provides such specifications as TR-196, which documents small cell synchronization and front-haul timing requirements. Moreover, ITU-T G.8271 Appendix VI includes a set of time error budgets for both network and equipment contributions. Vendors such as Microchip, ADI, and Calnex provide real-world jitter and holdover performance numbers along with recommended testing methodologies.

14.4 Typical Timing and Latency Targets for 5G RAN

The following table provides an overview of the set of parameters, the typical requirements, along with the associated sources.

Parameter	Typical Requirement	
Frequency Accuracy	+/-16 ppb	ITU-T G.8261 / G.8262
Time Error (Absolute)	Less than or equal to 15 microseconds	ITU-T G.8271 / 3GPP 38.104
Relative Cell-to-Cell Phase Error	Less than or equal to 3 microseconds (TDD);Less than or equal to 260 nanoseconds for 30 kHz SCS	3GPP TS 38.133
One-way Transport Latency (RU-to-DU)	Less than or equal to 100-250 microseconds	IEEE 1914.1 / O-RAN WG4
Packet Delay Variation	Less than 100 nanoseconds RMS	IEEE 1914.1 / ITU-T G.8261
PTP Message Interval	16/32 packets per second typical	IEEE 1588 / G.8275.1
SyncE Wander	Less than or equal to 0.1 UI p-p over 60 s	ITU-T G.8262

CHAPTER

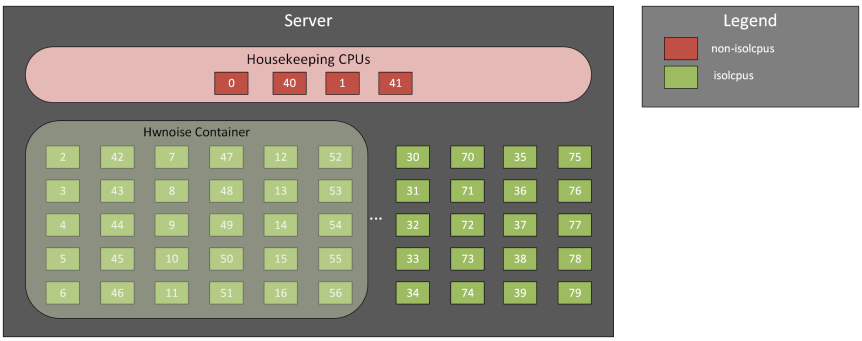
FIFTEEN

APPENDIX

The following section provides additional information on running the optional hwnoise as well as the timerlat test cases.

15.1 Hwnoise

The following figure presents a diagram of the hwnoise workload running inside a container on the DUT. Note that the given core assignment is for example purposes only. When necessary the core assignment should be adjusted to account for the specific CPU installed in the System Under Test.



To run the hwnoise test case as a container on the DUT:

```
apiVersion: v1
kind: Pod
metadata:
  name: hwnoise
  annotations:
    cpu-load-balancing.crio.io: "disable"
    irq-load-balancing.crio.io: "disable"
    cpu-quota.crio.io: "disable"
spec:
  # Map to the correct performance class in the cluster (from PAO)
  # Identify class names with "oc get runtimeclass"
  runtimeClassName: performance-openshift-node-performance-profile
  restartPolicy: Never
  containers:
  - name: rtla
    image: quay.io/container-perf-tools/rtla
    imagePullPolicy: Always
    resources:
```

(continues on next page)

(continued from previous page)

```
      requests:
        memory: "200Mi"
        cpu: "30"
      limits:
        memory: "200Mi"
        cpu: "30"
    env:
    - name: tool
      value: "rtla"
    - name: COMMAND
      value: "hwnoise"
    - name: DELAY
      value: "30"
    - name: DURATION
      value: "12h"
    - name: AA_THRESHOLD
      value: "30"
    - name: EXTRA_ARGS
      value: "--quiet"
    securityContext:
      # Required for access to /sys/kernel/debug on the host
      privileged: true
  nodeSelector:
    node-role.kubernetes.io/worker: ""
```

For the following test case:

1. Execute the hwnoise case against 15 cores (30 threads) on the DUT.
2. The test should run for a duration of 12 hours.
3. Once the test completes record the maximum single latency value across all cores along with the percentage of CPU availability across all cores.

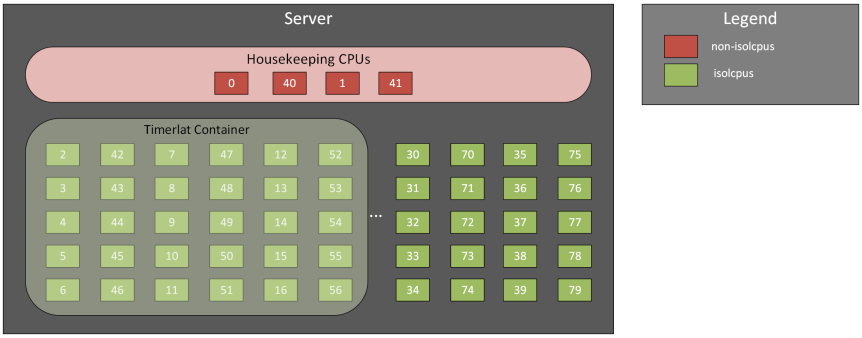
Note

- Adjust the threads assigned to the CPU assignment parameters as needed based on the CPU installed within the platform.
- It is not recommended to execute hwnoise against all available cores in the isolcpus list.
- The AA_THRESHOLD should be set to the threshold at which the test should be halted. Once the threshold is hit, a trace will be collected and an auto-analysis will be performed to help determine the cause of the failure. In the example above the test will be halted when the threshold of 30 microseconds has been reached.

15.2 Timerlat

The following figure presents a diagram of the timerlat workload running inside a container on the DUT. Note that the given core assignment is for example purposes only. When necessary the core assignment should be adjusted to account for the specific CPU installed in the System Under Test.

To run the timerlat test case as a container on the DUT:




```yaml
apiVersion: v1
kind: Pod
metadata:
  name: timerlat
  annotations:
    cpu-load-balancing.crio.io: "disable"
    irq-load-balancing.crio.io: "disable"
    cpu-quota.crio.io: "disable"
spec:
  # Map to the correct performance class in the cluster (from PAO)
  # Identify class names with "oc get runtimeclass"
  runtimeClassName: performance-openshift-node-performance-profile
  restartPolicy: Never
  containers:
  - name: rtla
    image: quay.io/container-perf-tools/rtla
    imagePullPolicy: Always
    resources:
      requests:
        memory: "200Mi"
        cpu: "30"
      limits:
        memory: "200Mi"
        cpu: "30"
    env:
    - name: tool
      value: "rtla"
    - name: COMMAND
      value: "timerlat"
    - name: DELAY
      value: "30"
    - name: DURATION
      value: "12h"
    - name: AA_THRESHOLD
      value: "30"
    - name: EXTRA_ARGS
      value: "--dma-latency 0"
    - name: EVENTS
      value: "sched,timer,irq_vectors,irq,nmi,ipi"
    securityContext:
```

(continues on next page)

(continued from previous page)

```
    # Required for access to /dev/cpu_dma_latency and /sys/kernel/debug on the host
    privileged: true
nodeSelector:
  node-role.kubernetes.io/worker: ""
```

For the following test case:

1. Execute the timerlat test case against 15 cores (30 threads) on the DUT.
2. The test should run for a duration of 12 hours.
3. Once the test completes record the maximum latency value along with the number of samples exceeding the threshold.

Note

- Similar to cylcictest it is not recommended to execute timerlat against all available cores in the isolcpus list.
- The AA_THRESHOLD should be set to the threshold at which the test should be halted. Once the threshold is hit, a trace will be collected and an auto-analysis will be performed to help determine the cause of the failure. In the example above the test will be halted when the threshold of 30 microseconds has been reached.